

useEffect and API Integration

Today You Will Learn

- What is useEffect?
- Dependency Array
- Component Lifecycle
- Fetch API
- Loading State
- Error Handling
- Build Real API Applications

📌 **Learning Outcome:** By the end of today's session, you will be able to fetch data from APIs, manage loading and error states, and build real-world React applications.

What is useEffect?

useEffect Hook

useEffect() is a React Hook used to perform side effects.

Side Effects

- Fetch API Data
- Call Backend APIs
- Access Local Storage
- Set Timers
- Add Event Listeners

Real World

- Load Products
- Get User Profile
- Weather Information
- Dashboard Data

Syntax

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Component Loaded");
}, []);
```

Dependency Array

The second argument of `useEffect` is called the **Dependency Array**.

Run Only Once

```
useEffect(() => {  
  console.log("Runs  
Once");  
}, []);
```

Run Every Render

```
useEffect(() => {  
  console.log("Runs Every  
Time");  
});
```

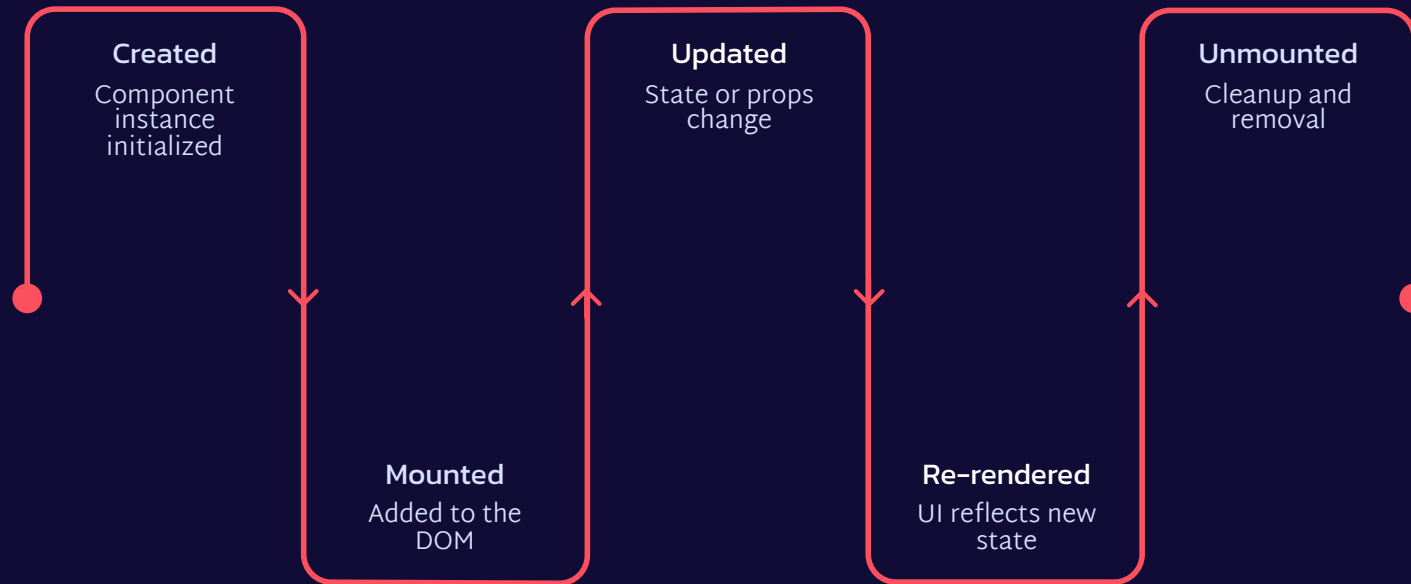
Run When Value Changes

```
useEffect(() => {  
  console.log("Count  
Changed");  
}, [count]);
```

📌 **Best Practice:** Always include the variables your effect depends on.

Component Lifecycle

React components go through different stages.



useEffect in Lifecycle

```
useEffect(() => {  
  console.log("Mounted");  
  return () => {  
    console.log("Unmounted");  
  };  
}, []);
```

Real World

- Load Data
- Clear Timers
- Remove Event Listeners
- Close WebSocket Connections

Fetch API

What is Fetch API?

Fetch API is used to request data from a server.

Example

```
useEffect(() => {  
  fetch("https://fakestoreapi.com/products")  
    .then(response => response.json())  
    .then(data => console.log(data));  
}, []);
```

Using Async/Await

```
const getProducts = async () => {  
  const response = await fetch(  
    "https://fakestoreapi.com/products"  
  );  
  const data = await response.json();  
  console.log(data);  
};
```

Real World APIs

Products

Users

Weather

Movies

Loading State

Show a loading message while waiting for the API response.

State & Fetch

```
const [loading, setLoading]
  = useState(true);

setLoading(true);
fetch(...)
setLoading(false);
```

Display

```
if (loading) {
  return <h1>Loading...</h1>;
}
```

📌 **Better User Experience:** Instead of a blank page, users know that data is loading.

Error Handling

Always handle API failures.

Try / Catch

```
const [error, setError]
  = useState("");

try {
  const response = await fetch(url);
  const data = await response.json();
} catch {
  setError("Something went wrong");
}
```

Display

```
if(error){
  return <h1>{error}</h1>;
}
```

Best Practice

Every API call should have:

Loading State

Success State

Error State

API Flow

How React Fetches Data



React Hooks Used

useState

useEffect

Mini Project: Product Listing App

Objective

Fetch products from an API and display them using Bootstrap Cards.

API

<https://fakestoreapi.com/products>

Features

- Fetch products using useEffect
- Store data using useState
- Show loading message
- Handle API errors
- Display products in responsive Bootstrap cards

Each Card Displays

- Product Image
- Product Title
- Price
- Category
- Buy Now Button

Concepts Used

- useEffect
- Fetch API
- useState
- Loading State
- Error Handling
- Bootstrap Grid

Homework Assignment

Random User Generator

API

<https://randomuser.me/api/>

Objective

Fetch one random user and display the information inside a Bootstrap Card.

Display

- User Photo
- Full Name
- Email
- Mobile Number
- Country
- Gender

Features

-  Fetch API using useEffect
-  Loading Message
-  Error Handling
-  Bootstrap Card
-  Responsive Layout
-  "Generate New User" Button to fetch another random user

Submission Checklist

 useEffect Hook

 Dependency Array

 Component Lifecycle

 Fetch API

 Loading State

 Error Handling

 Bootstrap Cards



Expected Outcome: Students will understand how to fetch data from external APIs using useEffect, manage asynchronous operations with loading and error states, and display dynamic data in responsive React applications. These concepts are fundamental for building real-world dashboards, e-commerce websites, weather apps, user management systems, and full stack applications.